

Smart Inventory & Expiry Management System

Comprehensive Project Report & Architecture Guide

1. Introduction

The Smart Inventory & Expiry Management System is a robust, desktop-based application designed to manage physical goods with strict expiry tracking. Built using **Python (Tkinter)** for the frontend and **MySQL** for the backend, the system addresses the critical problem of inventory spoilage and waste.

Unlike simple inventory trackers that store a single "total quantity" per product, this system tracks inventory at the **batch level**. This ensures accurate cost tracking and enables automated **FIFO (First-In, First-Out)** sales processing, ensuring oldest stock is sold first.

2. Core Operational Mechanics

2.1 The Batch-Level Concept

When 100 boxes of Paracetamol arrive, they are logged as a unique "Batch" with its own specific Manufacture Date, Expiry Date, and Cost Price. The system's inventory table tracks the *remaining quantity of that specific batch*.

2.2 Automated FIFO Sales

When processing a sale, the user only specifies the Product and Quantity. The system automatically searches for the oldest, non-expired batches of that product. If a sale requires 50 units, and the oldest batch only has 30 units, the system drains the oldest batch, and intelligently pulls the remaining 20 units from the *next* oldest batch.

2.3 Background Expiry Sweeping

The application features a non-blocking daemon thread that wakes up every day at midnight. It sweeps the database for any batches where the expiry date has passed. It moves these items to a permanent `expiry_log` (an audit trail of waste) and automatically zeros out their inventory, ensuring they cannot be accidentally sold.

3. File & Module Structure Breakdown

The codebase is organized into a modular, highly maintainable structure containing **22 distinct files**.

3.1 Database Layer (/database/)

The foundation of the application relies on strict ACID compliance and server-side logic.

- schema.sql**: Creates the database and 7 strictly-typed tables (categories, suppliers, products, batches, inventory, sales, expiry_log). It establishes Foreign Keys to prevent orphaned data.
- triggers.sql**: Contains 3 crucial database triggers. For example, `trg_before_sale` executes natively on the MySQL server to outright block any attempt to sell an expired batch or drop inventory below zero.
- procedures.sql**: Contains stored procedures like `sp_fifo_sale` (the complex cursor loop that handles multi-batch sales) and `sp_check_expiry`. Moving this logic to the database ensures operations are perfectly atomic and immune to network crashes.
- sample_data.sql**: A massive script containing roughly 200 rows of realistic pharmacy-themed seed data used for testing edge cases.
- er_diagram.md**: The visual Entity-Relationship Diagram documenting exactly how tables connect.

3.2 Python Backend Modules (/modules/)

These files handle business logic and safely interface with MySQL using parameterized queries to prevent SQL Injection.

- db_config.py (Root)**: The central connection hub containing the MySQL credentials. Imported by all modules.
- stock_manager.py**: Handles adding new products, suppliers, and processing incoming stock batches. Pulls live inventory levels.
- sales_manager.py**: Bridges the UI to the `sp_fifo_sale` stored procedure. Generates historical sales data and calculates precise profit margins per transaction.
- reports.py**: Houses complex aggregation queries. Calculates total inventory valuation across thousands of batches, and identifies low-stock items based on their reorder thresholds.
- expiry_monitor.py**: Manages the identification of batches expiring within 7 days, and generates KPIs regarding total wasted stock.
- scheduler.py**: A threading wrapper using the `schedule` library. It isolates the automated midnight expiry checks onto a secondary CPU thread so the graphical interface never freezes or stutters.
- error_handler.py**: intercepts generic MySQL `SIGNAL SQLSTATE` errors and maps them to clean, readable Python exceptions that the UI can gracefully display.

3.3 Graphical User Interface (/gui/)

The visual frontend is built with Python's Tkinter, using a customized, modern dark-mode palette (#1e1e2e background, #89b4fa accents).

- main.py (Entry Point)**: The top-level execution script. It performs a mandatory "ping" to the MySQL database; if the database is offline, it halts and shows a helpful troubleshooting guide.
- app.py**: The main window frame. It defines the universal color dictionaries, widget styling, and manages the central `Notebook` containing the 5 application tabs.
- dashboard_tab.py**: The overview screen. Displays dynamic KPI numbers and a highly visible warning table for stock expiring within the week.
- inventory_tab.py**: Displays all physical stock across the system. It houses the secure forms for inputting new stock (+ Add Batch), creating categories, suppliers, and product definitions.
- sales_tab.py**: The Point-of-Sale interface. Staff input the product and total quantity. The tab communicates with `sales_manager.py` to process the FIFO deduction, then displays the live-updated chronological history of all revenue and profit.
- expiry_tab.py**: An accountability screen. It offers filtering tools to predict upcoming expiration risks and displays the un-editable `expiry_log` detailing exact financial losses from spoiled goods.
- reports_tab.py**: An analytics dashboard featuring 4 interconnected tables: Low Stock Alerts, Total Inventory Valuation, aggregate Sales Summaries, and Waste Summaries.

4. Step-by-Step Flow of Execution

A highly technical walk-through of the system's smartest feature: A Sale.

- User Intent**: The user navigates to `sales_tab.py`, selects "Paracetamol 500mg", types 100 units, and hits "Process Sale".

2. **Frontend Validation:** Tkinter casts the inputs to `int` and `float`. If valid, it passes them to `modules/sales_manager.py`.
3. **Database Connection:** `sales_manager.py` requests a connection from `db_config.py`.
4. **Stored Procedure Execution:** A call is made to the MySQL procedure `sp_fifo_sale`.
5. **MySQL Cursor Initialization:** Inside the database server, a `CURSOR` fetches all active batches of "Paracetamol 500mg", ordering them stringently by `expiry_date ASC`.
6. **Transactional Loop:** For every batch fetched:
 - It checks the requested 100 units against the batch's available quantity.
 - It subtracts as much as possible, inserting a distinct row into the `sales` table for that specific batch.
 - *Note:* Before `sales` accepts the insert, the `trg_before_sale` Trigger wakes up to double-check that this particular batch hasn't secretly expired and has enough stock.
 - Once the insert succeeds, the `trg_after_sale` Trigger wakes up and automatically updates the `quantity_available` inside the `inventory` table.
7. **Commit & Respond:** Once all 100 units are fulfilled, MySQL `COMMITs` the transaction atomically and returns a success payload to Python.
8. **UI Refresh:** `sales_tab.py` catches the success, clears the input form, and triggers an app-wide `.refresh()`, ensuring the Dashboard and Reports instantly reflect the exact new stock totals and revenue numbers.